

Lecture 17: Consistency Models

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison

Consistency: Spectrum of Guarantees

Strong Consistency

- Linearizability — single-copy semantics; ops ordered by real time (Raft)
- Sequential consistency — total order; program order per client (ZooKeeper)

Weaker Consistency

- Causal — causally-related ops seen in the same order
- Eventual — all replicas converge, no ordering guarantee (Dynamo + Bayou)

Why the range?

- Strong → simpler for apps, but expensive (replicate to majority, wait for disk)
- Weak → better performance / availability, harder to reason about

Today: Linearizability, Sequential, Causal, Eventual, CAP Theorem

Later: Implementations!

Why Do Different Consistency Models Occur?

Fault tolerance requires copies

- Replicas on different nodes → operations on different copies
- Different clients may see different state



Key tension

- Keeping all replicas in sync has cost (latency, availability loss)
- Relaxing synchrony improves performance but weakens guarantees

Focus today: replicated, non-transactional storage

- Single read/write operations on individual data items
- Not full ACID transactions across multiple objects

Why Study Consistency Models?

Application developers: understand the contract

- What behavior can you rely on?
- How do you use the system correctly?

System designers: know what to optimize

- Different contracts enable different performance trade-offs
- Given a contract, design the system with the best possible performance

Perspective: what is observed by clients?

- Focus on a single data item (key-value store, register)
- Multiple concurrent clients issuing reads and writes

Other Worlds of Consistency (Not Today's Focus)

Hardware memory consistency

- How threads on different cores observe shared DRAM
- E.g., x86 Total Store Order, ARM relaxed model

Client-side caching consistency

- NFS, AFS: when do cached file copies become stale?

Transactions in databases

- Isolation levels: serializable, repeatable read, read committed, ...

Crash consistency

- What on-disk state is valid after a crash?

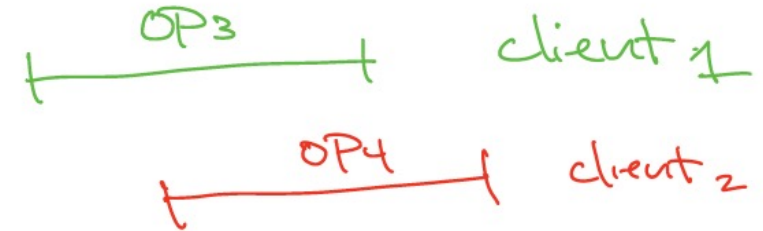
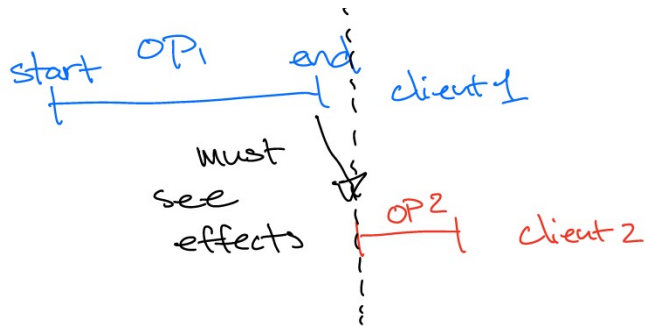
Linearizability (Herlihy & Wing, 1991)

Strongest consistency model for individual objects

- All ops appear to execute atomically at a single instant in time
- All clients see the same global ordering of operations

Real-time ordering constraint

- If op a completes before op b begins $\rightarrow$ a is ordered before b
- If a and b overlap in time $\rightarrow$ concurrent; either ordering is valid



Intuition: single-copy semantics

- Replication is completely hidden from clients
- Reads always return the most recently written value

Used by: Raft (after commit), Chain Replication (reads at tail)

Checking Linearizability

Each operation has an invoke time and a response time

- Operation's "atomic point" must fall within its [invoke, response] interval

Procedure

- Find an ordering of all operations consistent with real time
- Check each read returns the value of the most recent prior write in that order

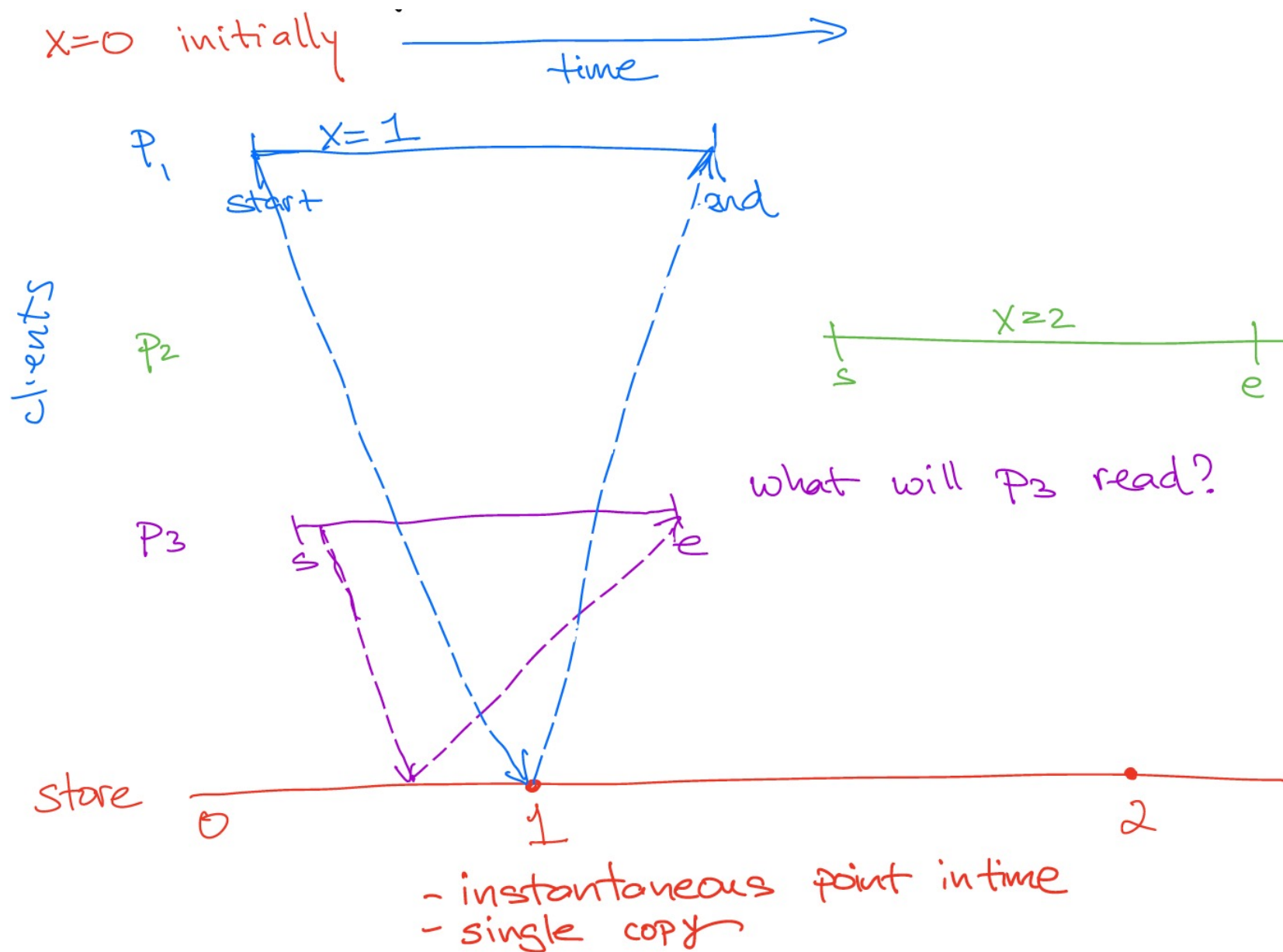
Two cases

- Non-overlapping: ordering is forced by real time
- Overlapping (concurrent): any ordering is permitted — find one that satisfies reads

Examples: processes $P_1 \dots P_6$ writing and reading register x (initially 0)

- Which ops are concurrent? Is there a valid linearization?
- What must each read return?

Linearizability Example 0



Linearizability: Example 1

PA $\overline{w(x=1)}$

PB

$\overline{w(2)}$

PC

$\overline{w(3)}$

PD

$\overline{w(4)}$ $\overline{w(5)}$

PE

$\overline{w(6)}$

PF

$\overline{r_{1?}}$ $\overline{r_{2?}}$ $\overline{r_{3?}}$ $\overline{r_{5?}}$ ~~$\overline{r_{6?}}$~~

Which are
concurrent?

Linearizability: Example 2

P_A $\overline{w(x=1)}$

P_B $\overline{w(2)}$

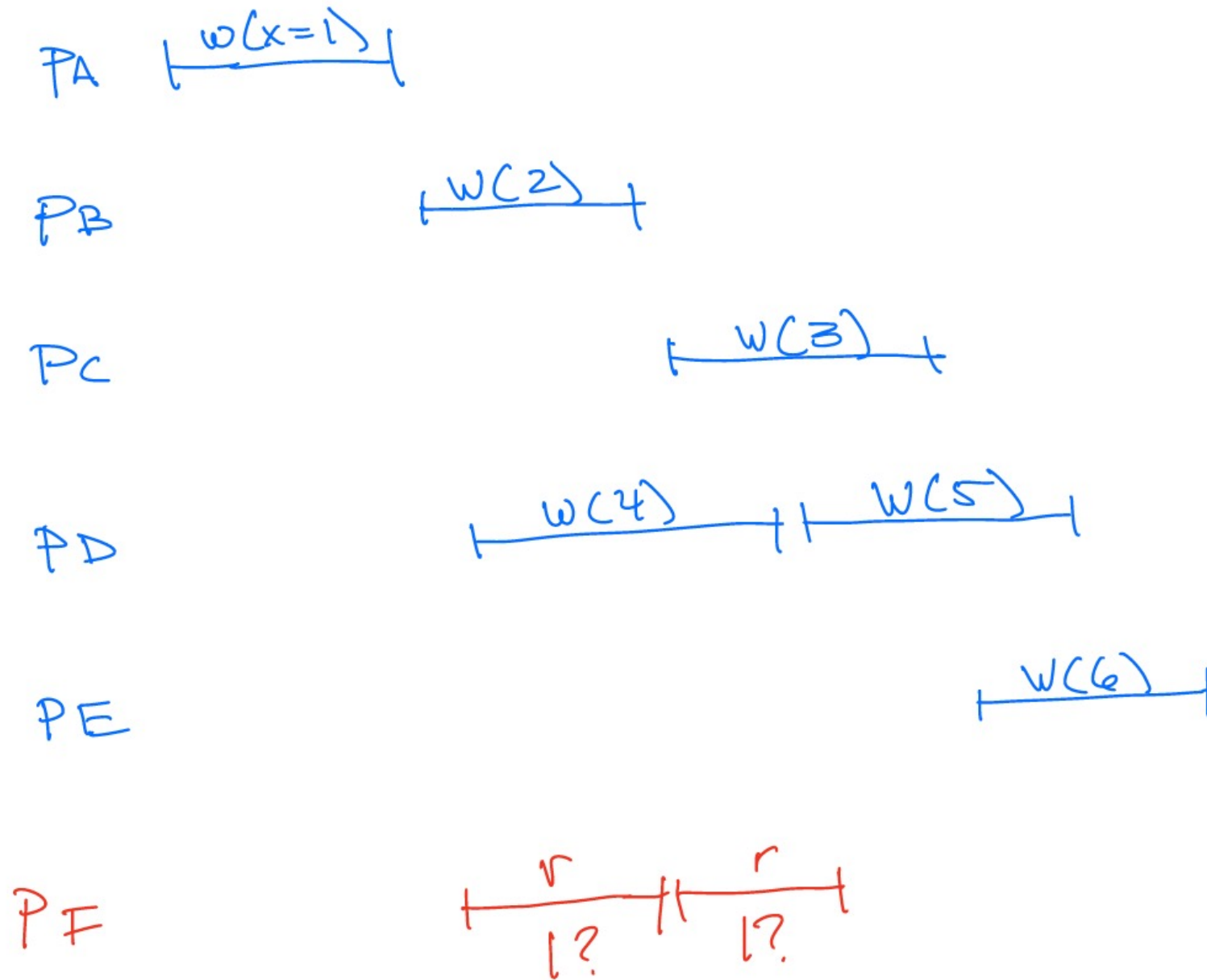
P_C $\overline{w(3)}$

P_D $\overline{w(4)}$ $\overline{w(5)}$

P_E $\overline{w(6)}$

P_F $\overline{r_{1?}}$ $\overline{r_{2?}}$ $\overline{r_{3?}}$ $\overline{r_{6?}}$ $\overline{r_{5?}}$

Linearizability: Example 3



Linearizability: Example 4

PA $\overline{w(x=1)}$

PB $\overline{w(2)}$

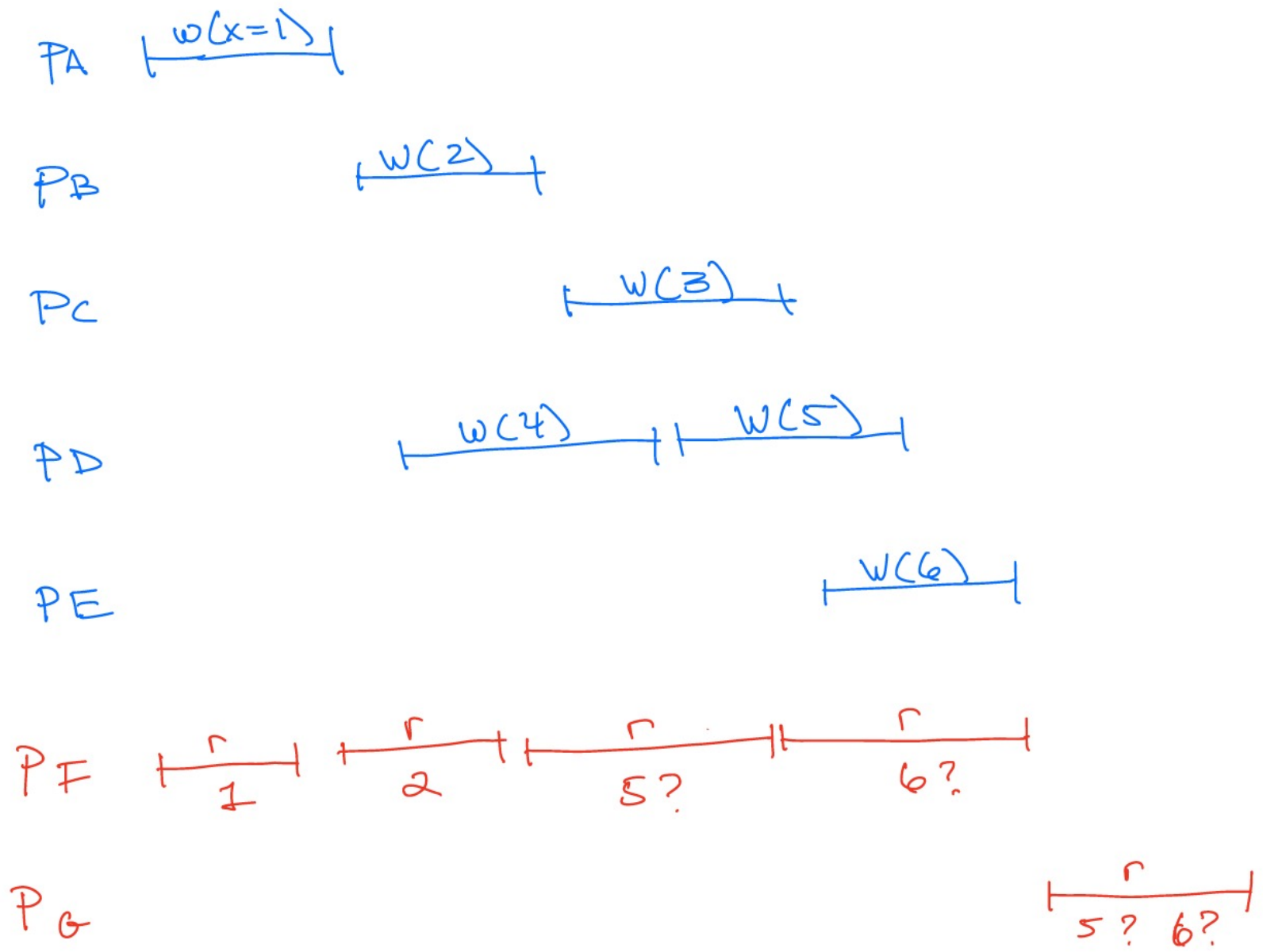
PC $\overline{w(3)}$

PD $\overline{w(4)} \parallel \overline{w(5)}$

PE $\overline{w(6)}$

PF $\overline{1} \parallel \overline{4?} \parallel \overline{2?} \parallel \overline{3?} \parallel \overline{6?}$

Linearizability: Example 5



Linearizability: Summary

Appears to be a single machine

- Single-copy semantics; replication hidden from clients
- Still provides fault tolerance underneath

Benefits

- Simplest model to reason about for application developers
- No surprising stale reads; every read is current

Costs

- Poor performance: must replicate to a majority and wait for disk before replying
- Reduced availability: majority quorum required even for reads in many designs

Eventual Consistency

Opposite extreme from linearizability

Guarantee

- Each replica eventually receives all writes
- If no new writes occur, all replicas converge to the same final value

Reads

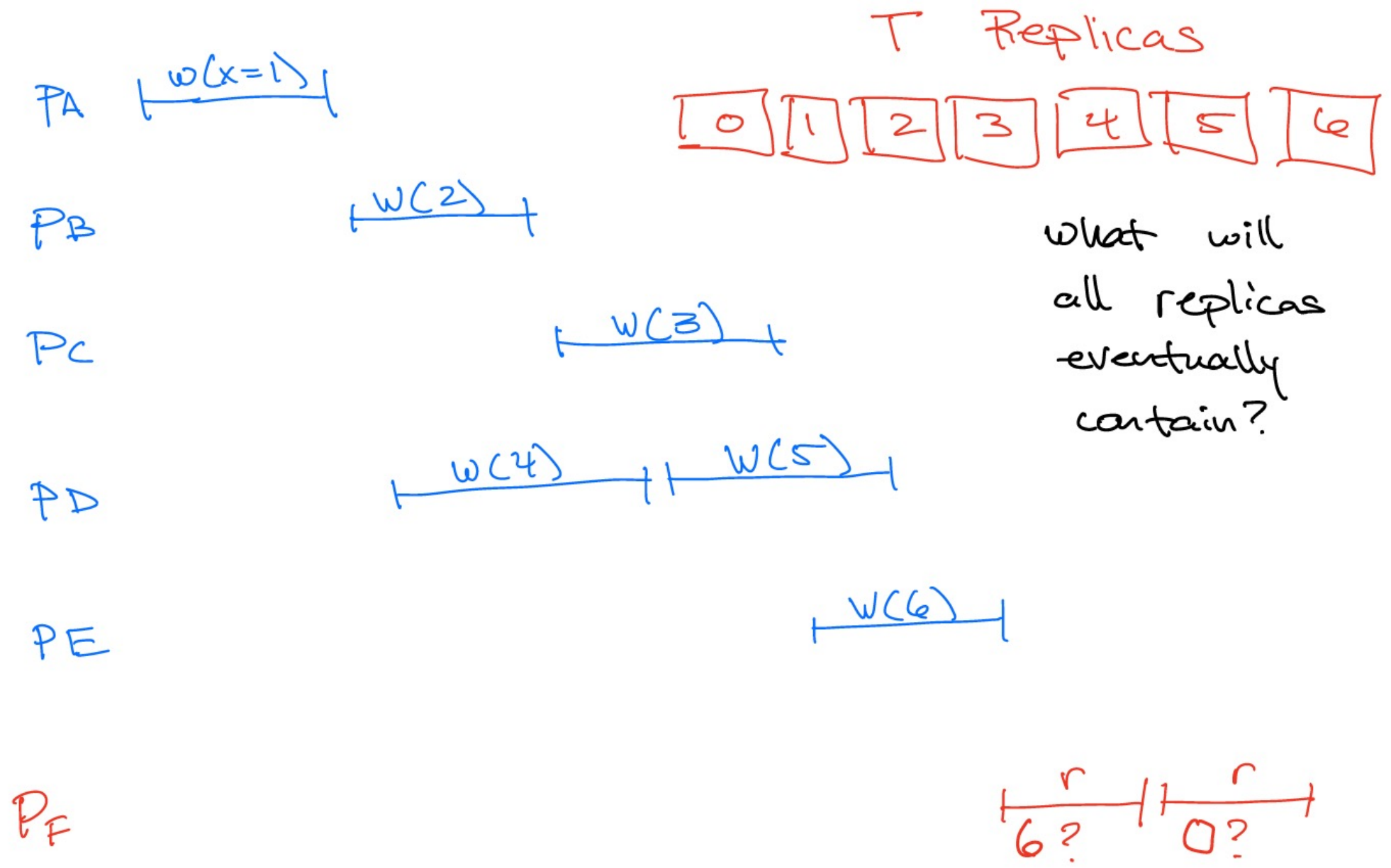
- May return any previously written value
- No guarantee of seeing the most recent write
- Stale reads are permitted indefinitely (until convergence)

Benefits

- Highest availability and lowest latency: write locally, propagate later

Example systems: Amazon Dynamo, Cassandra (default), Bayou

Eventual Consistency: Example



Sequential Consistency (Lamport, 1979)

Total order, but NOT constrained by real time

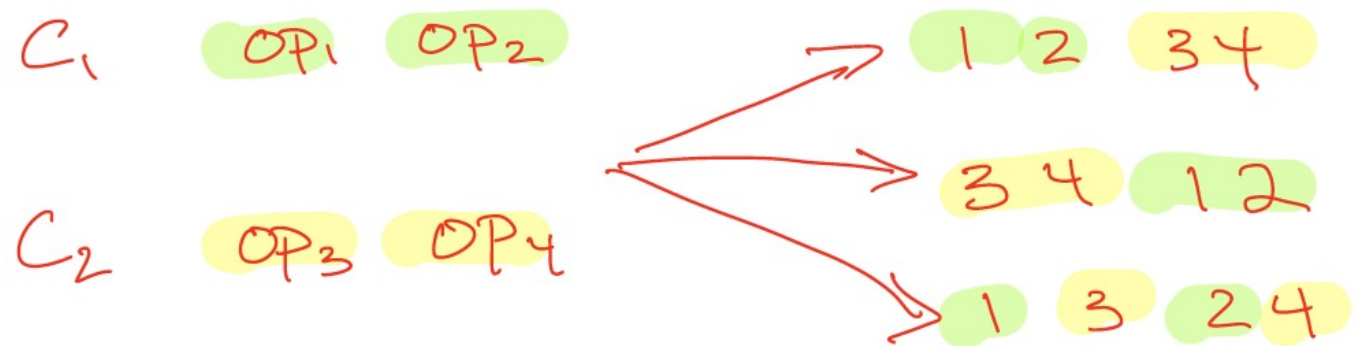
- All operations are totally ordered (agreed upon by all clients)
- Each client's operations appear in program order within that total order
- Total order need NOT match real time (unlike linearizability)

Comparison to linearizability

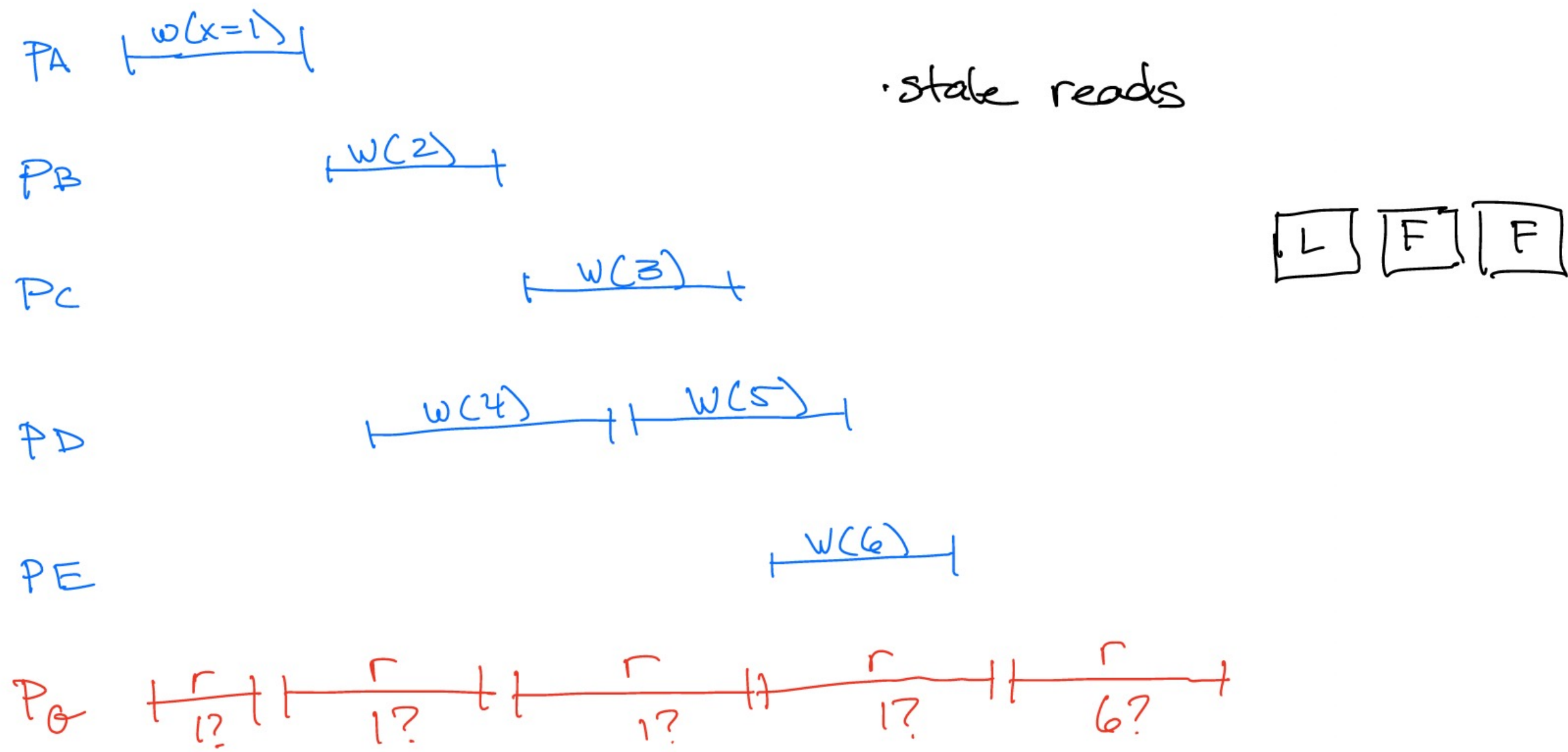
- Linearizability $\supset$ Sequential (linearizability is strictly stronger)
- Sequential consistency allows “stale” reads as long as some valid order exists

Example with 2 clients

- Actual times don't matter



Sequential Consistency: Example 1



Sequential Consistency: Example 2

PA $\overline{w(x=1)}$

PB $\overline{w(2)}$

PC $\overline{w(3)}$

PD $\overline{w(4)}$ $\overline{w(5)}$

PE $\overline{w(6)}$

PG $\overline{r_1?}$ $\overline{r_1?}$ $\overline{r_5?}$ $\overline{r_4?}$

Sequential Consistency: Example 3

PA $\overline{w(x=1)}$

PB $\overline{w(2)}$

PC $\overline{w(3)}$

PD $\overline{w(4)}$ $\overline{w(5)}$

PE $\overline{w(6)}$

PG $\overline{r_1?}$ $\overline{r_2?}$ $\overline{r_3?}$ $\overline{r_2?}$

Sequential Consistency: Example 4

PA $\overline{w(x=1)}$

PB $\overline{w(2)}$

PC $\overline{w(3)}$

PD $\overline{w(4)}$ $\overline{w(5)}$

PE $\overline{w(6)}$

PG $\overline{r_{1?}}$ $\overline{r_{1?}}$ $\overline{r_{3?}}$ $\overline{r_{2?}}$

Sequential Consistency: Example 5

P_A $\overline{w(x=1)}$

P_B $\overline{w(2)}$

P_C $\overline{w(3)}$

P_D $\overline{w(4)}$ $\overline{w(5)}$

P_E $\overline{w(6)}$

P_G $\overline{r_1?}$ $\overline{r_1?}$ $\overline{r_3?}$ $\overline{r_2?}$ same as prev. slide

P_H $\overline{1}$ $\overline{2}$ $\overline{3}$ $\overline{4}$ $\overline{3?}$

Sequential Consistency: Summary

Valid sequential execution

- Reads may return stale values as long as there exists a program-order-respecting total ordering across all clients that is satisfied

Invalid: violates program order

- A client's own operations must appear in order in the global sequence
- If a client writes $x=1$ then $x=2$, no client may observe $x=2$ then $x=1$

Key difference from linearizability

- Different clients may observe writes in different orders
- Sequential consistency does NOT guarantee real-time ordering across clients

Causal Consistency



No global total order — only causally-related ops are ordered

Causally related ($\rightarrow$ happens-before)

- Ops from the same client (program order)
- A write W is causal predecessor of read R if R observes W 's value
- Transitivity: if $A \rightarrow B$ and $B \rightarrow C$ then $A \rightarrow C$

Guarantee

- All clients observe causally-related ops in the same order
- Concurrent (causally unrelated) ops may be seen in different orders by different clients

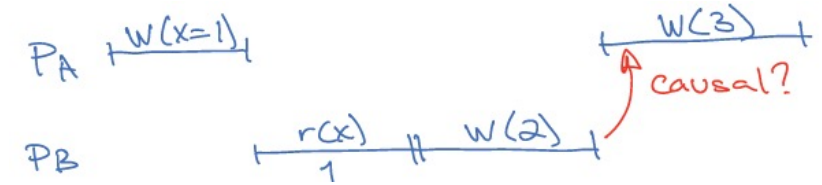
Relationship to other models

- Sequential $\supset$ Causal $\supset$ Eventual

Causal Consistency: Example

Scenario

- P_1 writes $x=1$; P_2 reads $x=1$, then writes $x=3$
- P_2 's write $x=2$ is causally after P_1 's write $x=1$



Causally consistent

- All clients must see $x=1$ before $x=2$ (causal order preserved)

NOT causally consistent

- A client observes $x=2$ and then $x=1$ (causal order violated)

Concurrent writes (no causal relationship)

- Two clients independently write $x=2$ and $x=3$ (no prior read)
- Different clients may see 2,3 or 3,2 — both valid under causal consistency
- Sequential would require one agreed-upon order for all clients

Capturing Out-of-Band Communication

Scenario

- 1. Alice updates her social media post (write to storage system)
- 2. Alice tells Bob via phone: “Check out my post!” (out-of-band)
- 3. Bob looks at Alice’s page (read from storage system)

What consistency does Bob need to see the post?

- Linearizable: yes — real-time ordering guarantees Bob sees the write
- Sequential: not necessarily — ordering may not reflect real time
- Causal: not necessarily — phone call is out-of-band, not tracked by system
- Eventual: not guaranteed — Bob may see a very stale version

Lesson

- Causal consistency only tracks causality visible to the storage system
- External (out-of-band) communication is invisible → need linearizability

Models Between Linearizability and Eventual

Monotonic reads

- Once you read a value, subsequent reads never return an older value

Consistent prefix

- Reads see a prefix of the write history — no gaps

Read my writes (session consistency)

- A client always sees the effects of its own prior writes

Monotonic writes

- A client's writes are applied in order across all servers

Bounded staleness

- Reads are at most k versions (or T seconds) behind

Real systems combine these

(e.g., Azure Cosmos DB session = MR + RMW)

Monotonic Reads

Guarantee

- Once a client reads a value, subsequent reads never return an older value
- “Time never goes backwards” within a session

Example: visitor counter

- Sequence of increments: 0, 0, 1, 0, 1, 0, 1, 3, 1, ...
- Client reads 3 → all subsequent reads by that client must return ≥ 3

Allowed read sequence

- $1 \rightarrow 3 \rightarrow 6 \rightarrow 7$ (non-decreasing)

NOT allowed

- $1 \rightarrow 3 \rightarrow 1$ (returned to 1 after seeing 3)

Implementation:

Client tracks highest version seen; servers reject stale responses

Consistent Prefix

Guarantee

- Reads see a **snapshot** corresponding to some prefix of the write history

	1	2	3	4	5	6	7	total
visitors	0	0	1	0	1	0		2
home	1	0	1	1	0	2		5

Example

- Consistent prefix: reads see 2-3, 0-1 (valid prefix)
- NOT consistent prefix: 2-1 (skipped past some writes)

Comparison with monotonic reads

- MR: values non-decreasing per client; does not prevent gaps
- CP: does not prevent seeing an older prefix
- MR and CP are orthogonal — neither implies the other

Read My Writes (Session Consistency)

Guarantee

- A client always sees the effects of its own prior writes
- Reads “follow” the client’s own write timeline

Example: visitor counter

- Writer updates counter to 5 → all subsequent reads by that writer return ≥ 5
- Other clients are not bound by this guarantee

Useful for

- Social feeds (user sees their own post immediately)
- Shopping carts, profile updates

Implementation:

Sticky sessions, version tokens, read-after-write quorums

Bounded Staleness

Guarantee

- Reads are at most k versions (or T seconds) behind the latest write
- A controlled relaxation of linearizability — some bounded lag is acceptable

Example: counter, staleness bound = 2 versions

- If current version is 5, any read must return $\geq$ version 3

Benefits

- Allows replicas to lag slightly $\rightarrow$ better throughput and availability
- Applications can reason about worst-case staleness

Relationship to other models

- Implies consistent prefix (you always see a valid prefix)
- Does NOT imply monotonic reads
(routing may go to a less-current replica within the bound)

CAP Conjecture and Theorem

CAP Conjecture — Eric Brewer, keynote at PODC 2000

- Popular interpretation: “Pick 2 of 3: Consistency, Availability, Partition Tolerance”

CAP Theorem — Gilbert & Lynch, 2002 (formal proof)

Definitions

- C: every read returns the most recent write (linearizability)
- A: every non-failing node can return a response to every request
- P: system operates correctly despite arbitrary network partitions

Key insight

- Network partitions will happen in any real distributed system
- So the real choice is: during a partition, sacrifice C or sacrifice A?
- CA (no partition tolerance) is not a practical design option

CAP: No Partitions

When there are no network partitions

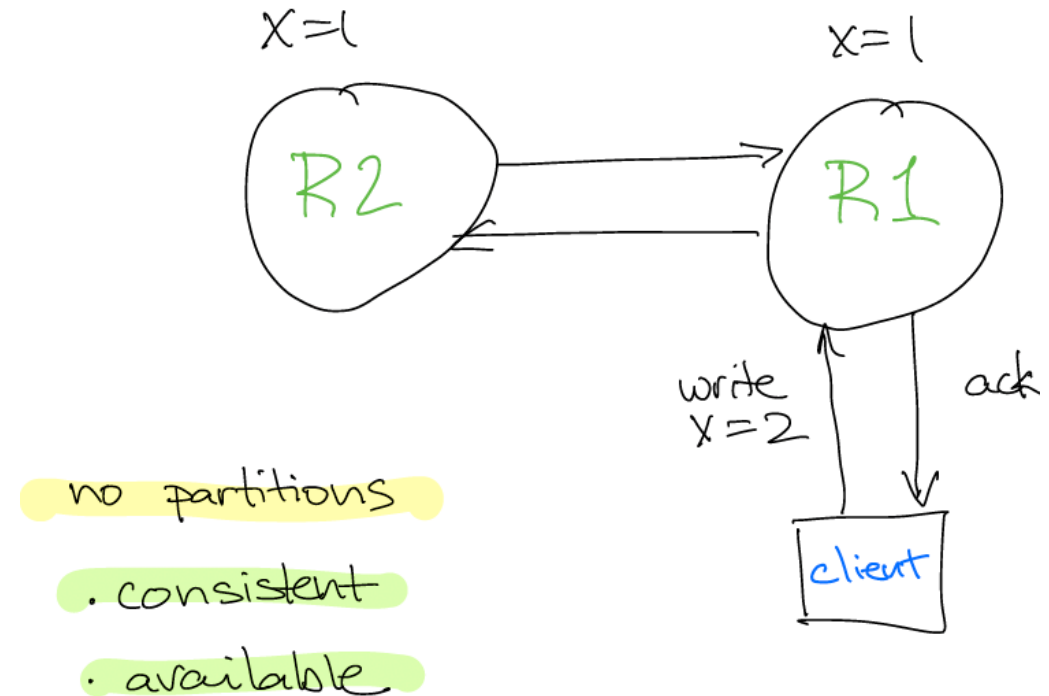
- Both C and A are achievable simultaneously
- Replicas can always communicate → can maintain linearizability and still respond

Cannot design a system that “chooses” no partitions

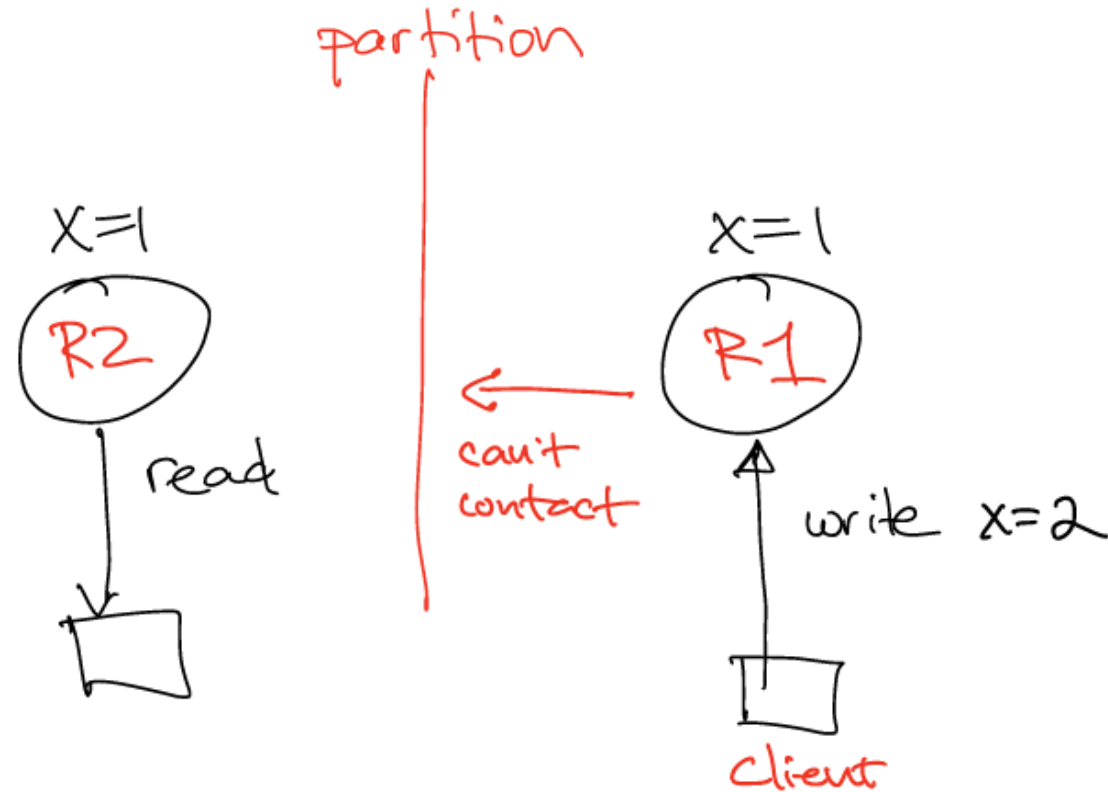
- Network will partition at some point (link failures, congestion, ...)
- Must decide what to do when partition occurs

Practical implication

- Meaningful design choice is AP vs. CP, not CA



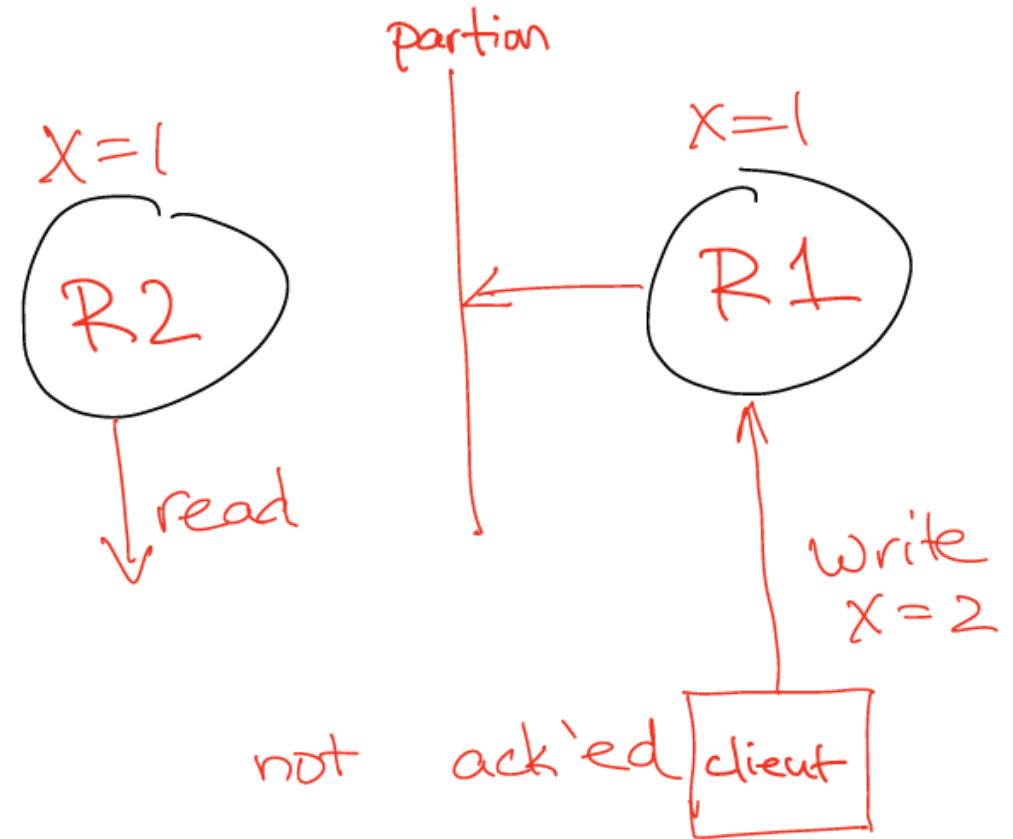
CAP: AP



AP — Available + Partition tolerant

- System responds to every request even during a partition
- May return stale / inconsistent data (sacrifice C)
- Examples: Amazon Dynamo, Cassandra, CouchDB

CAP: Consistency



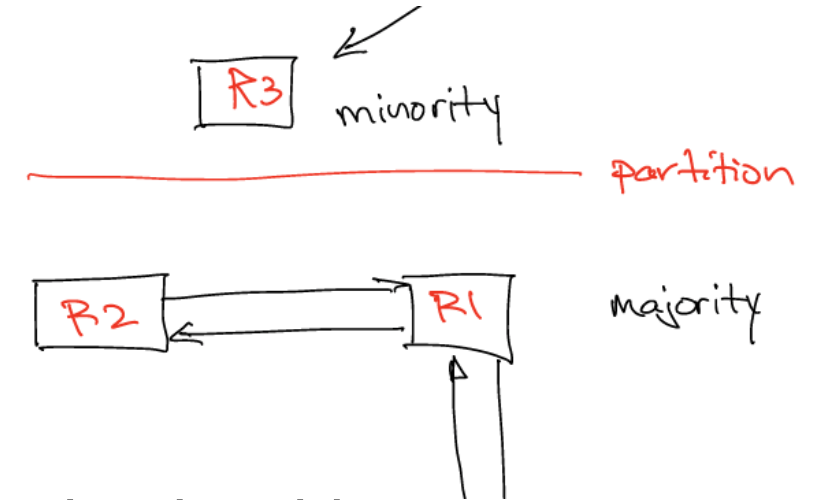
CP — Consistent + Partition tolerant

- System maintains linearizability during a partition
- May refuse to respond if it cannot reach a quorum (sacrifice A)
- Examples: Raft-based systems (ZooKeeper, etcd, Consul)

Reality is nuanced

- Availability is a spectrum (what fraction of requests fail?)
- PACELC extends CAP: trade-offs also exist under no partition (Latency vs. Consistency)

Raft and CAP



Scenario

- Network partition splits cluster into majority-side and minority-side
- Minority side cannot contact the current leader (or cannot form a quorum)

Raft chooses CP

- Minority-side clients get no response (requests time out)
- Majority side continues with the leader — remains consistent (linearizable)
- After partition heals: minority nodes catch up via log replication

Not necessarily available to all clients

- Minority-side clients are unavailable until partition heals

Summary: Consistency Models

Linearizability

- single-copy; real-time ordered; expensive but simple

Sequential consistency

- total order; program order per client; no real-time

Causal consistency

- causally-related ops ordered; concurrent ops may diverge

Eventual consistency

- all replicas converge; reads may be arbitrarily stale

Session models

- Monotonic reads, read my writes, consistent prefix, bounded staleness
- Combine for “good enough” guarantees at lower cost

CAP Theorem

- Pick CP or AP during a partition — partition tolerance is non-negotiable

Next:

Eventual Consistency in Bayou; Consistency Measurement at Facebook